Reviewer Pack — Pseudoexpectation Spectral Gap Lower-Bounds SoS Refutation D…

Entry f781296ddc7c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 22:55:04 UTC

Pseudoexpectation Spectral Gap Lower-Bounds SoS Refutation Degree on 3-XOR

Entry ID: `f781296ddc7c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-26 22:55:04 UTC

1. Conjecture

Field A (mathematical branch): Sum-of-Squares hierarchy via degree-2 pseudoexpectation moment matrices (Lasserre/Parrilo SDP relaxations) **Field B** (complexity object): Sum-of-Squares refutation degree of random 3-XOR CSP instances

Statement:

> For an unsatisfiable 3-XOR instance F on n variables with m clauses, let M(F) be the n×n centered second-moment matrix whose (i,j) entry is sgn_ij = (#clauses where x_i,x_j co-occur with equal parity sign) − (#with opposite parity sign), and let g(F) = (λ_max(M) − λ_2(M))/λ_max(M) be its normalized spectral gap. Then SoS refutation degree d_SoS(F) satisfies d_SoS(F) ≥ ⌈2/g(F)⌉ + 2 whenever F is unsatisfiable, and the slack d_SoS(F) − ⌈2/g(F)⌉ is bounded above by O(log n). Equivalently, a single instance with g(F) ≥ 0.5 and d_SoS(F) = 4 falsifies the lower bound.

Rationale (proposer's reasoning):

> SoS refutations of k-XOR are governed by spectral expansion of the clause-variable incidence (Grigoriev/Schoenebeck), but the specific gap of the *signed parity co-occurrence* matrix has not been isolated as a degree predictor distinct from second-eigenvalue-of-incidence bounds. A tight gap-to-degree law would explain why mildly structured (non-expander) 3-XOR is refutable at low degree while expanding 3-XOR forces Ω(n) degree, sharpening the Grigoriev expansion lower bound into a quantitative two-sided estimate. This avoids relativization (it inspects the formula's matrix directly) and does not invoke pseudorandomness, so it dodges the natural-proofs barrier.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `46bc6d9f4e0c6198`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 11 (n)×4 (m/n)×50 seeds = 2200 UNSAT 3-XOR instances, the conjecture is SUPPORTED iff (a) every instance satisfies d_SoS(F) ≥ ⌈2/g(F)⌉+2, and (b) the per-n mean slack s̄(n)=mean(d_SoS−⌈2/g⌉) satisfies s̄(n) ≤ 3·log2(n) for all n∈{6,...,16}.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Sum-of-Squares refutation degree random 3-XOR spectral` - `Lasserre hierarchy pseudoexpectation 3XOR lower bound moment matrix` - `SoS degree lower bound random CSP spectral gap refutation`

Top relevant hits considered: - [http://arxiv.org/abs/1605.00058v2] Strongly Refuting Random CSPs Below the Spectral Threshold - [http://arxiv.org/abs/1208.6015v1] The spectral function of a first order elliptic system - [http://arxiv.org/abs/1009.0997v3] Spectral asymptotics for Robin problems with a discontinuous coefficient - [http://arxiv.org/abs/2101.05167v1] A Sublevel Moment-SOS Hierarchy for Polynomial Optimization - [http://arxiv.org/abs/1209.3049v1] Lower bounds on the global minimum of a polynomial - [http://arxiv.org/abs/1602.05409v2] Lasserre Lower Bounds and Definability of Semidefinite Programming - [http://arxiv.org/abs/1701.04521v1] Sum of squares lower bounds for refuting any CSP - [http://arxiv.org/abs/1911.02911v1] Extended Formulation Lower Bounds for Refuting Random CSPs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import json
from itertools import combinations

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
```

```
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1]
        for j in range(i+1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]

    return x

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def compute_g(F):
    n = len(F)
    M = [[0] * n for _ in range(n)]

    for i, j in combinations(range(n), 2):
        count_equal = 0
        count_opposite = 0
        for clause in F:
            if (clause[i] == clause[j]):
                count_equal += 1
            else:
                count_opposite += 1
        M[i][j] = M[j][i] = count_equal - count_opposite

    # Compute eigenvalues using numpy's eigh for symmetric matrices
    import numpy as np
    eigenvalues = np.linalg.eigvalsh(M)
    lambda_max = max(eigenvalues)
    lambda_2 = sorted(eigenvalues)[-2]

    if lambda_max == 0:
        return float('inf')  # Avoid division by zero

    g = (lambda_max - lambda_2) / lambda_max
    return g

def compute_d_SoS(F):
    n = len(F)
    m = len(F[0])
    variables = list(range(n))

    def is_satisfiable(cert):
```

```
        for clause in F:
            if all(cert[i] == 1 - (clause[i] ^ 1) for i in range(m)):
                return True
        return False

    max_degree = 6
    for degree in range(2, max_degree + 1, 2):
        for comb in combinations(variables, degree):
            cert = [0] * n
            for var in comb:
                cert[var] = 1
            if is_satisfiable(cert):
                return degree

    return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [6, 8, 11, 14]
    m_over_n_values = [1.0, 2.0, 4.0, 8.0]
    instances_tested = 0
    total_slack = 0

    for n in n_values:
        for m_over_n in m_over_n_values:
            m = int(n * m_over_n)
            F = [[random.randint(0, 1) for _ in range(m)] for _ in range(n)]

            # Ensure the instance is unsatisfiable
            while True:
                if not any(all(F[i][j] == 1 - (F[i][k] ^ 1) for k in range(m)) for j in range(m)):
                    break

            g = compute_g(F)
            d_SoS = compute_d_SoS(F)

            instances_tested += 1
            total_slack += max(0, d_SoS - math.ceil(2 / g))

    conjecture_holds = all(d_SoS >= math.ceil(2 / g) + 2 for n in n_values for m_over_n in m_over_
    counterexample = "" if conjecture_holds else "g≥0.5, d_SoS=4"

    return {
        "metric_name": "slack",
        "metric_value": total_slack / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_slack = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std=0.0 support_fraction={support_fraction}")
    elif any(d_SoS == 4 and g >= 0.5 for r in results for d_SoS, g in zip(r["d_SoS"], r["g"])):
        first_failing_seed = next(seed for seed, result in enumerate(results) if any(d_SoS == 4 ar
        print(f"RESULT: FALSIFIED counterexample=\"g≥0.5, d_SoS=4\" first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f172745.py", line 140, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f172745.py", line 115, in run_trial
    if not any(all(F[i][j] == 1 - (F[i][k] ^ 1) for k in range(m)) for j in range(m)):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f172745.py", line 115, in <genexpr>
    if not any(all(F[i][j] == 1 - (F[i][k] ^ 1) for k in range(m)) for j in range(m)):
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f172745.py", line 115, in <genexpr>
    if not any(all(F[i][j] == 1 - (F[i][k] ^ 1) for k in range(m)) for j in range(m)):
                     ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test script crashed with a NameError before producing any data, so neither the support nor falsification conditions could be evaluated. No empirical evidence exists for or against the conjecture. | next: Fix the buggy generator (the 'i' variable is out of scope in the nested generator at line 115) and rerun the 2200-instance sweep to obtain actual d_SoS and spectral-gap measurements.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	19192
2	preregistration	claude_max	opus	0	0	6432
3	novelty	claude_max	opus	0	0	2757
4	novelty	claude_max	opus	0	0	11295
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37836
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15123
7	judge	claude_max	opus	0	0	4107

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 96744 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/f781296ddc7c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f781296ddc7c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f781296ddc7c.tar.gz` (if generated)